

Data Science

From a raw file to a deployed model at scale

A 16-Session Course

What this course covers

Part A — Getting hold of data

- S1 Introduction to Data Science
- S2 Data Collection & Pre-Processing

Part B — Making data fit to use

- S3 Data Cleaning
- S4 Data Integration
- S5 Data Transformation
- S6 Data Reduction & Discretization

Part C — Understanding data

- S7 Descriptive Statistics
- S8 EDA & Visualization

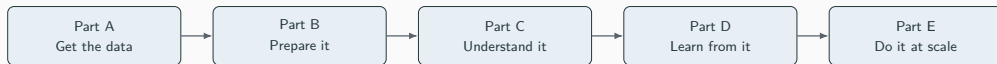
Part D — Learning from data

- S9 Supervised Learning
- S10 Unsupervised Learning & Clustering
- S11 Model Evaluation
- S12 Model Saving & Deployment

Part E — Data at scale

- S13 Big Data Fundamentals
- S14 Distributed Data Processing
- S15 NoSQL & Visualization at Scale
- S16 Capstone Project

How the parts build on each other



You cannot model data you have not prepared, and you cannot prepare data you have not understood. Everything in this course follows that order.

Part I

Part A — Getting Hold of Data

Session 1 — Introduction to Data Science

Data science turns recorded facts into decisions.

Three ingredients have to come together:

Ingredient	What it gives you	Without it
Programming	handle more rows than a person could ever read	stuck with what fits on one screen
Mathematics	tell a real pattern from a coincidence	find patterns in noise
Domain knowledge	know what the numbers <i>mean</i>	correct answer to the wrong question

The seven-stage lifecycle



Stage 3 is most of the work

Beginners expect modelling to take longest. In practice, collecting and cleaning the data consumes most of a real project — modelling is often an afternoon.

Three kinds of data

The shape of your data decides how much work is needed before you can analyse it.

Kind	Looks like	Examples	Effort
Structured	rows and columns, every row the same shape	CSV, SQL tables	Low
Semi-structured	has labels, but rows can differ	JSON, XML, logs	Medium
Unstructured	no rows or labels at all	text, images, audio	High

Analogy — the master chef

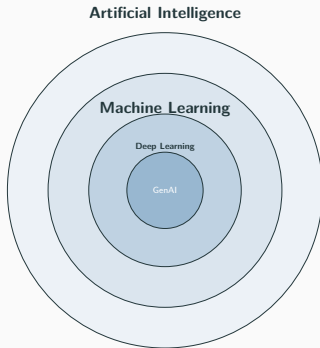
Raw ingredients are raw data. Knives and ovens are programming tools. Recipes are mathematical methods. The plated dish is the decision somebody acts on.

How the field grew

Era	Hard part	Typical method
Statistics (pre-2000)	collecting data at all	averages, hypothesis tests, by hand
Big data (2000–2010)	storing and querying it	databases, Hadoop, SQL
Machine learning (2010–now)	choosing what to learn	algorithms find the pattern
Generative AI (2020–now)	steering the output	very large models

Early on, a human proposed the pattern and tested it with data. Now the machine proposes the pattern and the human tests it.

AI, machine learning, deep learning, generative AI



The useful distinction: **who writes the rule?** In ordinary programming, you do. In machine learning, the machine works out the rule from examples.

Who does the work

Role	Owns	Tools	Delivers
Data engineer	getting data in, reliably	SQL, Python, cloud	a trustworthy table
Data analyst	explaining what happened	spreadsheets, dashboards	a report
Data scientist	predicting what will happen	Python, scikit-learn	a model

Analogy — the car factory

The data engineer builds the assembly line. The data analyst counts defects coming off it. The data scientist works out which settings cause fewer defects next month.

Where you meet it every day

- **Recommendations** — Netflix, Amazon, Spotify
- **Dynamic pricing** — airlines, ride-hailing
- **Fraud detection** — your bank, every card swipe
- **Medical diagnostics** — radiology
- **Demand forecasting** — retail stock
- **Predictive maintenance** — factories, railways
- **Route optimisation** — maps, delivery
- **Credit scoring** — lenders

A prediction without a range is a guess in a suit

A number reported without its uncertainty invites a decision it cannot support.

Session 2 — Data Collection & Pre-Processing

Five ways to collect data

Route	Use when	Watch out for
Internal databases	facts about your own operations	undocumented schemas
Files handed to you	someone already extracted it	unknown extraction method
APIs	the owner is happy to share	rate limits, format drift
Web scraping	no API, but public	terms of service
Sensors / surveys	the data does not exist yet	slow and expensive

Sampling: how a sample lies

Method	How it picks	Risk
Random	every row, equal chance	needs the full list
Stratified	proportionally from each group	must know the groups
Systematic	every n th row	fails on repeating patterns
Convenience	whatever was easy	usually biased, invisibly

More data does not fix a biased sample

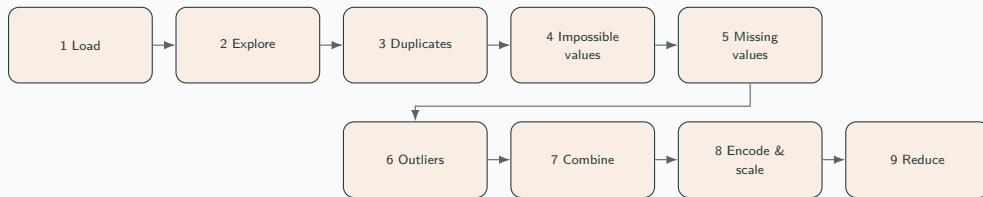
Doubling a biased sample gives a more confident wrong answer. Only changing *how* you collect fixes bias.

Preprocessing is turning raw data into a table an algorithm can read. It is most of the work in most projects.

Raw data is messy because it is produced by people and imperfect machines:

- somebody typed their age as 225
- a sensor dropped offline for four hours
- one system writes dates one way, another writes them differently
- a customer pressed *Submit* twice
- “Mumbai”, “mumbai”, “Bombay” and “MUM” are the same city

The order we clean in



Two orderings are not negotiable

Impossible values before missing values — an age of 225 must become “missing” first. Scaling comes last, only after the data is split for testing.

Pandas: the spreadsheet that takes orders

Analogy — a spreadsheet you talk to

Instead of dragging to select rows, you say “give me the rows where city is Mumbai” — and it works the same whether there are 20 rows or 20 million.

Five commands answer almost everything about a new table:

Command	Answers
<code>.shape</code>	how big is it?
<code>.head()</code>	what do the rows look like?
<code>.info()</code>	what type is each column, how much is missing?
<code>.describe()</code>	what are the numeric columns like?
<code>.value_counts()</code>	what values does a text column hold?

Key takeaways — Sessions 1–2

- Data science is a **sequence of stages**, not one clever trick.
- The three ingredients — programming, mathematics, domain knowledge — must all be present.
- Structured, semi-structured and unstructured data cost very different amounts of preparation.
- A biased sample cannot be fixed by collecting more of it.
- Cleaning and preparation, not modelling, is where most project time goes.

Part II

Part B — Making Data Fit to Use

Session 3 — Data Cleaning

Six cleaning jobs, one fixed order



Impossible is not the same as extreme

Impossible = it cannot be true (age of 225) — turn it into missing, then impute it. Extreme = unusual but real (a 4,00,000 purchase) — think hard before touching it.

Filling missing values

Analogy — the missing puzzle piece

An imputed value is a good guess standing in for a blank. In data science, guessing wrong can ruin a model's accuracy.

Situation	Fill with
Numbers, roughly symmetric	mean
Numbers, skewed or with extremes	median
Text or categories	mode (most common value)
Groups that genuinely differ	the group's own median
More than ~40% missing	drop the column

Noise is small random error on top of a real pattern — not wrong data, just distraction.

Over-smoothing erases the thing you were looking for

A wide averaging window removes noise beautifully but also arrives late at every real peak — and a genuine spike (fraud, a failure) can vanish entirely. The window is a trade between noise and lag; no setting avoids both.

Detecting outliers: two rules

Rule	Flags a value if	Use when
IQR rule	below $Q_1 - 1.5 \text{ IQR}$ or above $Q_3 + 1.5 \text{ IQR}$	almost always
Z-score rule	more than 3 std. deviations from the mean	data is roughly bell-shaped

In fraud data, the outliers are the answer

A pipeline that automatically removes outliers deletes every fraud case you were hired to find — and reports excellent accuracy on what is left.

Session 4 — Data Integration

Five ways to integrate

Approach	Good	Bad
Physical (warehouse)	fast queries, one place to look	stale, expensive
Virtual (federation)	always current	slow, breaks if a source is down
Application-based	no central system	every app must know every other
Middleware	apps stay simple	middleware is a single point of failure
Manual	works today	slow, unrepeatable, error-prone

Analogy — the wedding guest list

Two partners keep guests in different formats. Integration is the painful merge into one list — and deciding whether “R. Sharma” and “Raj Sharma” are the same person is the whole difficulty.

Stacking vs linking

Operation	Question it answers	Function
Stacking	"here are more rows of the same thing"	concat
Linking	"here is extra info about the same things"	merge

**Two months of sales: stack. Customers and their orders:
link.**

The four join types

Type	Keeps	Use when
inner	only rows matching in both	you need complete records
left	all left rows, blanks where unmatched	the left table is your population
right	all right rows	rare, usually a relabelled left
outer	everything from both	looking for what does not match

A join that changes the row count is a claim

Shrinking says some records had no match. Growing says a key is not unique. Always check the row count before and after.

Session 5 — Data Transformation

Many algorithms measure distance between rows. A column with big numbers dominates that distance, whatever it means.

Analogy — judging a contest in different units

One judge scores out of 10, another out of 1,000. Add the scores and the second judge decides the winner every time — not because their opinion is better, but because their scale is bigger.

Five transformers

Tool	Produces	Works on
MinMaxScaler	values between 0 and 1	columns
StandardScaler	mean 0, std. dev. 1	columns
RobustScaler	centred on median, scaled by IQR	columns
Normalizer	each row rescaled to length 1	rows
Binarizer	0 or 1, split at a threshold	columns

Which algorithms actually need scaling

Distance-based (kNN, K-Means, SVM) and neural nets: yes. Tree-based models (decision trees, random forests): no — a tree asks “is age > 30?”, and units don’t change that split.

Kind	Meaning	Encode with
Nominal	no order (city, colour)	one-hot
Ordinal	a real order (small < medium < large)	ordinal encoding
The target	what you predict	label encoding

Never label-encode a nominal feature

Label-encode city as Delhi=0, Mumbai=1, Pune=2 and the algorithm believes Pune is “twice” Mumbai. None of that is true.

The train–test split, and leakage

Analogy — practice papers and the real exam

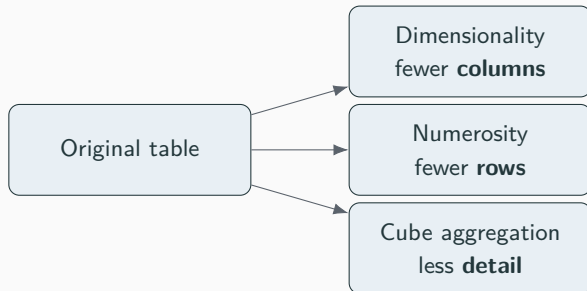
The training set is practice papers with answers. The test set is the exam. A model tested on its own training data is a student marking their own homework.

The test set is spent after one use

Every peek at the test score while choosing a model leaks a little information into your choices. Split off the test set, do all tuning by cross-validation on the training data, and look at the test set once.

Session 6 — Data Reduction & Discretization

Three kinds of reduction



Feature selection: three families

Family	How it decides	Example
Filter	score each column vs. target, alone	correlation, chi-square
Wrapper	retrain repeatedly, add/remove columns	recursive feature elimination
Embedded	the model selects while training	Lasso, tree importance

Analogy — the shadow

Hold a teapot to a lamp; the shadow is 2-D, yet you can still tell it is a teapot, because you chose the angle that keeps the most information. Where it breaks down: you cannot tell which side the handle is on.

PCA costs you the ability to explain the model

After PCA, inputs are “component 1”, “component 2”. If asked “why was this loan refused?”, that answer means nothing to the applicant.

Discretization: top-down vs bottom-up

Family	Starts from	Example
Top-down (splitting)	the whole range, one bin	equal-width, equal-frequency
Bottom-up (merging)	every value its own bin	chi-merge

Bin when the categories mean something to a human. Do not bin just to tidy up — binning always destroys information.

Key takeaways — Sessions 3–6

- Clean in a fixed order: duplicates, impossible values, missing values, noise, outliers, text/types.
- Check the row count after every join — silently doubled or shrunk data is the costliest undetected error.
- Split before you scale; never label-encode a nominal feature.
- Reduction always throws something away — know what, and be able to justify it.

Part III

Part C — Understanding Data

Session 7 — Descriptive Statistics

Mean, median, mode

Measure	What it is	Answers
Mean	add everything, divide by count	if shared equally, how much each?
Median	sort, take the middle value	what does the middle person have?
Mode	most frequent value	what is most common?

Analogy — the pub and the billionaire

Ten people in a pub average 5 lakh in wealth. A billionaire walks in — the average leaps past 90 crore, while the median barely moves. Which number describes “how rich are the people in this pub”?

Spread: four measures

Measure	Computed as	Sensitive to outliers?
Range	$\max - \min$	extremely
Variance	average squared distance from mean	yes
Std. deviation	$\sqrt{\text{variance}}$	yes
IQR	$Q_3 - Q_1$, the middle half	no

Analogy — two archers

Both archers average dead centre. One puts every arrow within a coin's width of the bullseye; the other scatters corner to corner. Identical averages, only one can actually hit a target.

Skewness	Shape	Mean vs median
≈ 0	symmetric	equal
positive	long tail right	mean > median
negative	long tail left	mean < median

A quick check needing no calculation: compare the mean and the median. If the mean is noticeably higher, the data is right-skewed.

$$z = \frac{\text{value} - \text{mean}}{\text{standard deviation}}$$

A z-score says how many standard deviations a value sits from the mean — so you can compare things measured on completely different scales.

Most real data is not normally distributed

Financial, biological and web-traffic data routinely break the bell-curve assumption. Check before you assume: plot a histogram, compare mean and median, look at kurtosis.

Session 8 — EDA & Visualization

The first pass: five questions in order

#	Question	Tool
1	How big is it, what type is each column?	<code>.shape</code> , <code>.info()</code>
2	What does each column look like alone?	histogram, box plot
3	What is missing or impossible?	<code>.isna().sum()</code>
4	Which columns move together?	scatter, correlation
5	What differs between groups?	<code>groupby</code> , <code>pivot</code>

Analogy — the doctor's examination

A doctor does not begin with surgery. Most checks find nothing — that is the point — and occasionally one finds something that changes the whole plan.

Choosing the chart

Question	Chart
How is one number spread out?	histogram
Centre, spread, outliers?	box plot
How many of each category?	bar chart
Do two numbers move together?	scatter plot
Does a number differ by group?	box plot by group
How do many numeric columns relate?	heatmap

Correlation only measures straight-line relationships

A curved but perfect relationship can score a Pearson correlation near zero. Look at the scatter plot, not just the number.

A correlation is a question worth investigating, never an answer.

Three things a correlation can mean:

1. A causes B
2. B causes A
3. **C causes both** — usually this one (a confounder)

Analogy — ice cream and drowning

Ice cream sales correlate strongly with drowning deaths. Ice cream does not cause drowning — hot weather causes both.

Pivot tables, and checking the count behind an average

Always check the counts behind an average

An average computed from two rows sits in a pivot table looking exactly as authoritative as one computed from two hundred. Print the count table alongside any pivot of means.

Key takeaways — Sessions 7–8

- Never report a centre without a spread — and look at the shape too.
- Compare mean vs median as a free, instant skew check.
- Every chart should answer a question you asked *before* drawing it.
- Correlation is a lead to investigate, not a finding to report.

Part IV

Part D — Learning From Data

Session 9 — Supervised Learning

Supervised learning trains an algorithm on historical data where the answer is already known.

Analogy — flashcards

The front of the card is the input (features). The back is the answer (label). Learning is guessing, flipping the card, and adjusting — until you get it right every time.

Regression or classification?

	Target is	Examples
Regression	a number	house price, tomorrow's temperature
Classification	a category	spam or not, which disease

A coefficient is the effect of that feature while every other feature is held constant.

“Each extra bedroom adds 4 lakh” is wrong. **“Each extra bedroom adds 4 lakh, comparing houses of the same area and age”** is right — and a completely different claim.

Three regression metrics

Metric	What it is	Character
MAE	average size of the error	treats all errors evenly
RMSE	square root of avg. squared error	punishes large errors harder
R^2	share of variation explained	1.0 perfect, 0 = no better than the mean

Always compare against a baseline

R^2 of 0 means "no better than always guessing the average." Run the baseline before trusting any score.

Session 10 — Unsupervised Learning & Clustering

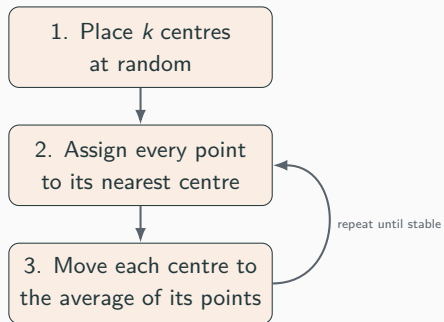
Clustering has no answer key

	Supervised	Unsupervised
Data	X and y	X only
Can you measure accuracy?	yes	no
Fails by	predicting badly	producing groups that mean nothing

Analogy — sorting the laundry

Nobody told you the categories. Somebody else would make different piles — by fabric instead of colour. Neither is wrong. Several defensible answers, no marking scheme.

K-Means, in three steps



Scale first, always

K-Means measures straight-line distance. An unscaled column with big numbers decides every cluster on its own.

Method	Idea
Elbow method	inertia always falls with more clusters — look for the bend where it stops paying
Silhouette score	how much closer a point is to its own cluster than the next nearest — higher is genuinely better

Clustering always returns clusters

Ask K-Means for 4 groups in pure random noise and it will confidently return 4 groups. A tidy chart is not evidence that groups exist.

Session 11 — Model Evaluation

Overfitting and underfitting

	Underfitting	Overfitting
Model is	too simple	too complex
Training score	poor	excellent
Test score	poor	poor
It has	missed the pattern	memorised the noise

Analogy — three students revising

One skimmed a page (bad at practice and the exam). One memorised the practice papers word for word (perfect at practice, lost on the real exam). Only the good student, who learned the method, does well at both.

A single train-test split gives you one number, and that number depends on which rows happened to land in the test set.

Report a range, not a point

If single splits vary by 0.15 in score, and your model beats a colleague's by 0.03, you have shown nothing. Cross-validate and report the spread.

K-fold cross-validation

test	train	train	train	train
train	test	train	train	train
train	train	test	train	train
train	train	train	test	train
train	train	train	train	test

**Every row is tested exactly once. Report the average score
AND its spread across folds.**

The test set is spent after one use

Tuning on the test set invalidates the score

Every time you look at the test score and change something because of it, you leak information into your choices. The discipline: split off the test set, tune only with cross-validation on training data, look at the test set once, at the very end.

Session 12 — Model Saving & Deployment

A model saved without its scaler and encoder will accept raw input and return confident, wrong predictions — and nothing will raise an error.

Analogy — the recipe and the cooking

Training is developing the recipe (weeks of work). Saving is writing it down. Prediction is cooking (twenty minutes). You do not redevelop the recipe every time someone orders dinner.

Offline vs online deployment

	Offline (batch)	Online (live)
When it runs	on a schedule	the moment someone asks
Input/output	a whole file	one row / one answer
Needs a server always on?	no	yes
Good for	churn scores, restock lists	loan decisions, fraud checks

Most projects should start offline. If a nightly file of predictions solves the problem, you have avoided servers, uptime and latency entirely.

What breaks after launch

Problem	What happens
Data drift	inputs shift over time
Concept drift	the relationship itself shifts
Schema change	an upstream column is renamed
Version drift	the library is upgraded under the model
Extrapolation	users enter values outside the training range

Validate at the door

A model will happily predict a price for a house of -50 square feet. The input layer, not the model, is where you catch nonsense.

Key takeaways — Sessions 9–12

- A coefficient's meaning always includes “holding everything else constant.”
- Clustering has no ground truth — justify the result, don't just trust the score.
- Cross-validate and report the spread; one split is one opinion.
- Save the whole pipeline; validate inputs; expect the world to drift after launch.

Part V

Part E — Data at Scale

Session 13 — Big Data Fundamentals

The five Vs

V	The problem it creates
Volume	does not fit in memory or on one disk
Velocity	cannot store it before the next batch arrives
Variety	many shapes need different handling
Veracity	how much can you trust it
Value	most of it is never used

Only one of these is usually your actual problem. A library with a volume problem needs a bigger building; one with a velocity problem needs a faster door.

“Big data” is not a size, it is a symptom — the point where one machine stops being enough. That point arrives far later than most people think.

Try these before you build a cluster

Chunking, selecting only needed columns, and fixing dtypes routinely turn a “we need Spark” problem back into a laptop problem. Spend a day on these first.

Four types of analytics



Increasing order of difficulty and value — descriptive is a groupby; prescriptive needs a prediction *and* a judgement call.

How HDFS works

Idea	What it means
Blocks	the file is cut into fixed-size pieces (~128 MB)
Replication	every block stored on three different machines
Write once	files are appended to, never edited in the middle

HDFS is terrible at small files

A million tiny files use a million entries in the NameNode's in-memory index — exhausting it long before the disks fill up. Combine small files into large ones before storing.

Session 14 — Distributed Data Processing

MapReduce: three steps

Step	Input	Output	Runs
Map	one chunk	many (key, value) pairs	in parallel
Shuffle	all pairs	grouped by key	across the network
Reduce	values for one key	one result per key	in parallel

Analogy — counting votes

A hundred counters each tally their own box (map). Tallies for each party are gathered together (shuffle). One person adds up each party's tallies (reduce) — tiny, because it works on tallies, not ballots.

Map and reduce run locally. The shuffle moves data between machines — hundreds of times slower than memory.

Not every reducer can be combined

sum, min, max and count are safe to pre-aggregate locally. average is not — the average of averages of unequal groups is wrong. Fix: carry (sum, count) through the shuffle.

Why Spark replaced Hadoop MapReduce

	Hadoop MapReduce	Apache Spark
Between stages	writes to disk	keeps data in memory
Multi-pass work	very slow	fast
Interactive queries	no	yes

One pass: barely any difference. Thirty passes — what training a model looks like — Spark wins outright.

Lazy evaluation

Kind	Examples	Runs immediately?
Transformation	filter, select, groupBy, join	no — recorded
Action	show, count, collect, write	yes — executes the plan

Lazy evaluation moves your errors

A typo in a column name may not surface until an action runs, possibly many lines later. When timing Spark code, time the action.

When not to use Spark

Use Spark when the data does not fit, not when it feels large

Spark pays a real cost before doing any work at all — roughly 7 seconds of pure startup and shuffle overhead. It does not earn that back on a 200 MB CSV, however important that CSV is.

Session 15 — NoSQL & Visualization at Scale

Why NoSQL exists

	Relational (SQL)	NoSQL
Shape decided	before you store anything	as you store
Scales by	a bigger machine	more machines
Best for	money, bookings	logs, profiles, catalogues

Analogy — the filing cabinet and the folders

A filing cabinet holds identical labelled forms — tidy, but a document that does not fit needs the whole form redesigned. A folder system holds whatever each case needs — nothing rejected, but you can no longer assume any two folders match.

The four NoSQL families

Family	Best at	Example
Document	flexible, nested records	MongoDB
Wide-column	huge volume, known queries	Cassandra
Key-value	caching, sessions	Redis
Graph	networks, recommendations	Neo4j

When a distributed database's machines cannot talk to each other, you must choose between a possibly stale answer and no answer.

This is a business decision, not a technical one

For a bank balance: refuse to answer rather than risk staleness — choose consistency. For a page view counter: a thirty-second-old count beats a blank page — choose availability.

Draw ten million dots and you get a solid rectangle.
Aggregate or bin before you plot — never send raw rows to the chart.

A sample must be random

`df.head(10000)` is not a sample — it is whatever was written first, usually ordered by time or ID. Use a genuine random sample, and label the margin of error.

Session 16 — Capstone Project

A finished project is a question, an honest answer, and a statement of what the answer cannot support.

Choosing the metric after seeing results is not evaluation, it is rationalisation

Write the target down before touching the data: e.g. “catch 60% of churners while contacting under 30% of customers.” Then judge the model against that number, not against 1.0.

Association rules: support, confidence, lift

Measure	Question	Formula
Support	how often does this happen at all?	$\text{count(both)} / \text{total}$
Confidence	when left happens, how often right?	$\text{support(both)} / \text{support(left)}$
Lift	more likely than chance?	$\text{confidence} / \text{support(right)}$

Confidence alone will fool you

A rule pointing at a very common item gets high confidence automatically. Lift near or below 1 means the rule found nothing.

A box plot showing two groups apart is a hypothesis, not a finding. A p-value puts a number on how likely the gap is chance.

A p-value is not the size of an effect

With enough rows, even a trivial difference produces a tiny p-value. Always report the effect size next to it — “churners make 1.4 more calls on average ($p < 0.001$)” is a finding; “ $p < 0.001$ ” alone is not.

Question	Before shipping
Who could be harmed by a wrong prediction?	name the expensive error
Does it treat groups differently?	check error rate per group, not just overall
Could you explain a decision to the person it affected?	if the answer is “component 1 was low,” reconsider

Key takeaways — Sessions 13–16

- Big data is a symptom, not a size — measure before reaching for a cluster.
- MapReduce is map, shuffle, reduce; the shuffle is the expensive part.
- NoSQL trades a fixed schema for flexibility — and CAP is a business choice.
- A finished project states plainly what its result cannot support.

Closing

Sixteen sessions, one sentence each

1. Data science turns facts into decisions
2. Load and look before you touch
3. Clean in a fixed order
4. Join carefully; check row counts
5. Split before you scale; never leak
6. Fewer columns is often better
7. Report centre, spread *and* shape
8. Every chart answers a question
9. Beat a baseline, always
10. Clustering has no answer key
11. One split is one opinion
12. Save the pipeline; watch for drift
13. Big data is a symptom, not a size
14. Map, shuffle, reduce
15. NoSQL trades schema for flexibility
16. State what your result cannot support

Thank you.